

CXGate
CCZGate
CDKMRippleCarryAdder
CHGate
CPhaseGate
CRXGate
CRYGate
CRZGate
CSdgGate
CSGate
CSwapGate
CSXGate
CU1Gate
CU3Gate
CUGate
CXGate
CYGate
CZGate
DCXGate
Diagonal
DiagonalGate
DraperQFTAdder
ECRGate
efficient_su2
EfficientSU2
evolved_operator_ansatz
EvolvedOperatorAnsatz
ExactReciprocal
excitation_preserving
ExcitationPreserving
fourier_checking
FourierChecking
FullAdderGate
FunctionalPauliRotations
GlobalPhaseGate
GMS
GR
GraphState
GraphStateGate
grover_operator
GroveOperator
GRX
GRY
GRZ
HalfAdderGate
hamiltonian_variational_ansatz
HamiltonianGate
HGate
hidden_linear_function
HiddenLinearFunction
HSCumulativeMultiplier
IGate
Initialize
InnerProduct
InnerProductGate
IntegerComparator
IQP
iqp
Isometry
iSwapGate
LinearAmplitudeFunction
LinearFunction
LinearPauliRotations
MCMT
MCMTGate
MCMTVChain
MCPhaseGate
MCXGate
MCXGrayCode
MCXRecursive
MCXVChain
ModularAdderGate
MSGate
MultiplierGate
n_local
NLocal
OR
OrGate
pauli_feature_map
pauli_two_design
PauliEvolutionGate
PauliFeatureMap
PauliGate
PauliTwoDesign
Permutation
PermutationGate
phase_estimation
PhaseEstimation
PhaseGate
PhaseOracle
PiecewiseChebyshev
PiecewiseLinearPauliRotations
PiecewisePolynomialPauliRotations
PolynomialPauliRotations
qaoa_ansatz
QAOAAnsatz
QFT
QFTGate
QuadraticForm
quantum_volume
QuantumVolume
random_bitwise_xor
random_iqp
RCCXGate
RCCXGate
real_amplitudes
RealAmplitudes
RGate
RQFTMultiplier
RVGate
RXGate
RXXGate
RYGate
RYYGate
RZGate
RZXGate
RZZGate
SdgGate
SGate
StatePreparation
SwapGate
SXdgGate
SXGate
TdgGate
TGate
TwoLocal
U1Gate
U2Gate
U3Gate
UCGate
UCPauliRotGate
UCRXGate
UCRYGate
UCRZGate
UGate
unitary_overlap
UnitaryGate
UnitaryOverlap
VBERippleCarryAdder
WeightedAdder
XGate
XOR
XXminusYYGate
XXPlusYYGate
YGate
z_feature_map
ZFeatureMap
ZGate
zz_feature_map
ZZFeatureMap
qiskit.circuit.singleton

Quantum information (qiskit.quantum_info)
Transpilation
Primitives and providers
Results and visualizations
Serialization (OpenQASM and QPY)
Pulse-level programming
Other
Release notes

CXGate

```
class qiskit.circuit.library.CXGate(*args, **kwargs):
    force_mutable=False, **kwargs)
```

GitHub ↗

Bases: [SingletonControlledGate](#)

Controlled-X gate.

Can be applied to a [QuantumCircuit](#) with the [cx\(\)](#) and [cnot\(\)](#) methods.

Circuit symbol:



Matrix representation:

$$CX_{q_0, q_1} = I \otimes |0\rangle\langle 0| + X \otimes |1\rangle\langle 1| = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}$$

Note

In Qiskit's convention, higher qubit indices are more significant (little endian convention). In many textbooks, controlled gates are presented with the assumption of more significant qubits as control, which in our case would be `q_1`. Thus a textbook matrix for this gate will be:

$$CX_{q_1, q_0} = |0\rangle\langle 0| \otimes I + |1\rangle\langle 1| \otimes X = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

In the computational basis, this gate flips the target qubit if the control qubit is in the $|1\rangle$ state. In this sense it is similar to a classical XOR gate.

$$|a, b\rangle \rightarrow |a, a \oplus b\rangle$$

Create new CX gate.

Attributes

base_class

Get the base class of this instruction. This is guaranteed to be in the inheritance tree of [cnot\(\)](#)

The "base class" of an instruction is the lowest class in its inheritance tree that the object should be considered entirely compatible with for `_all_` circuit applications. This typically means that the subclass is defined purely to offer some sort of programmer convenience over the base class, and the base class is the "true" class for a behavioral perspective. In particular, you should *not* override [base_class](#) if you are defining a custom version of an instruction that will be implemented differently by hardware, such as an alternative measurement strategy, or a version of a parametrized gate with a particular set of parameters for the purposes of distinguishing it in a [Target](#) from the full parametrized gate.

This is often exactly equivalent to [type\(obj\)](#), except in the case of singleton instances of standard-library instructions. These singleton instances are special subclasses of their base class, and this property will return that base. For example:

```
1 >>> isinstance(XGate(), XGate)
2 True
3 >>> type(XGate()) is XGate
4 False
5 >>> XGate().base_class is XGate
6 True
```

In general, you should not rely on the precise class of an instruction; within a given circuit, it is expected that [Instruction.name](#) should be a more suitable discriminator in most situations.

condition

The classical condition on the instruction.

⛔ Deprecated since version 1.3.0

The property `qiskit.circuit.instruction.Instruction.condition` is deprecated as of qiskit 1.3.0. It will be removed in 2.0.0.

condition_bits

Get Cbits in condition.

⛔ Deprecated since version 1.3.0

The property `qiskit.circuit.instruction.Instruction.condition_bits` is deprecated as of qiskit 1.3.0. It will be removed in 2.0.0.

ctrl_state

Return the control state of the gate as a decimal integer.

decompositions

Get the decompositions of the instruction from the `SessionEquivalenceLibrary`.

definition

Return definition in terms of other basic gates. If the gate has open controls, as determined from [ctrl_state](#), the returned definition is conjugated with X without changing the internal [definition](#)

duration

Get the duration.

⛔ Deprecated since version 1.3.0

The property `qiskit.circuit.instruction.Instruction.duration` is deprecated as of qiskit 1.3.0. It will be removed in Qiskit 2.0.0.

label

Return instruction label

mutable

Is this instance is a mutable unique instance or not.

If this attribute is [False](#) the gate instance is a shared singleton and is not mutable.

name

Get name of gate. If the gate has open controls the gate name will become:

`<original_name_o-ctrl_state>`

where `<original_name>` is the gate name for the default case of closed control qubits and `<ctrl_state>` is the integer value of the control state for the gate.

num_cbits

Return the number of cbits.

num_ctrl_qubits

Get number of control qubits.

Returns

The number of control qubits for the gate.

Return type

[int](#) ↗

num_qubits

Return the number of qubits.

params

Get parameters from base_gate.

Returns

List of gate parameters.

Return type

[list](#) ↗

Raises

CircuitError – Controlled gate does not define a base gate

unit

Get the time unit of duration.

⛔ Deprecated since version 1.3.0

The property `qiskit.circuit.instruction.Instruction.unit` is deprecated as of qiskit 1.3.0. It will be removed in Qiskit 2.0.0.

Methods

control

```
control(num_ctrl_qubits=1, label=None, ctrl_state=None, annotated=False)
```

GitHub ↗

Return a controlled-X gate with more control lines.

Parameters

- `num_ctrl_qubits` ([int](#) ↗) – number of control qubits.

On this page

CXGate
Attributes
base_class
condition
condition_bits
ctrl_state
decompositions
definition
duration
label
mutable
name
num_cbits
num_ctrl_qubits
num_qubits
params
unit
Methods
control
inverse

Was this page helpful?

Yes No

Report a bug or request content on GitHub ↗.

- **label** (*str* γ / *None*) – An optional label for the gate (Default: `None`)
- **ctrl_state** (*str* γ / *int* γ / *None*) – control state expressed as integer, string (e.g. `"110"`), or `None`. If `None` use all 1s.
- **annotated** (*bool* γ) – indicates whether the controlled gate should be implemented as an annotated gate.

Returns

controlled version of this gate.

Return type

ControlledGate

inverse

`Inverse(annotated=False)`

GitHub ↗

Return inverted CX gate (itself).

Parameters

annotated (*bool* γ) – when set to `True` this is typically used to return an `AnnotatedOperation` with an inverse modifier set instead of a concrete `Gate`. However, for this class this argument is ignored as this gate is self-inverse.

Returns

inverse gate (self-inverse).

Return type

CXGate